EP2 (7 enero 2025) **IC-24-25-Q1**

Apellidos: ……Nombre:……..Grupo:……DNI:……

Examen EP2 (temas 9-14)

Duración: 2 horas 30 minutos. La solución de cada ejercicio se tiene que escribir en el espacio reservado para ello en el propio enunciado. No podéis utilizar calculadora, apuntes, etc. La solución del examen se publicará en Atenea el 8 de enero por la tarde y las notas antes de las dos del 9 de enero.

Ejercicio 1 (2 puntos) El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección 0xB000 y justo a continuación la sección de código (.text).

```
.data
        N = 3
V1:     .word 0x0102, 0x0304, 0x0506
        .even
Res:    .space 2

.text
        MOVI    R0, lo(N)
        MOVHI   R0, hi(N)
        MOVI    R1, lo(V1)
        MOVHI   R1, hi(V1)
        MOVI    R2, lo(Sub)
        MOVHI   R2, hi(Sub)
        MOVI    R5, 0
Loop:   LDB     R7, 1(R1)
        JALR    R3, R2
        ADDI    R0, R0, -1
        ADDI    R1, R1, 2
        BNZ     R0, Loop
        STB     0(R1), R5
        …

Sub:
        ADD     R5, R5, R7
        JALR    R3, R3
.end
```

Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

a. ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? (binario - 0,25 cada dirección)

Loop = 0xB016 Res = 0xB006

b. ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? (binario - 0,25 cada instrucción)

Instrucción	@memoria y contenido
`MOVHI R1, hi(V1)`	Mem_w[0x B00E]=0x93B0
`BNZ R0, Loop`	Mem_w[0x B01E]=0x81FB

c. Una vez ejecutado el programa en el SISC Von Neumann ¿Cuál es la dirección de la memoria donde ha escrito la instrucción (STB 0(R1), R5) y cuál es su contenido? (binario – 0,5 puntos)

Mem_b[0x B006]=0x09

d. Una vez ejecutado el programa en el SISC Von Neumann ¿Cuántas instrucciones se han ejecutado del código que se muestra y cuántos ciclos ha tardado la ejecución de todo el código, desde la primera instrucción (MOVI R0, lo(N)) hasta la instrucción STB, ambas incluidas? (0,25 por las instrucciones, 0,25 por los ciclos)

N.º de instrucciones ejecutadas = 29

N.º de ciclos = 91

Apellidos: ……Nombre:……Grupo:……DNI:……

Ejercicio 2 (1 punto) Completa la siguiente tabla ensamblando las instrucciones en ensamblador SISA o desensamblando las instrucciones en lenguaje máquina según sea necesario. Indica poniendo NA en la casilla aquellos casos en los que la instrucción no corresponda al lenguaje SISA. (0,25 por cada instrucción correcta)

Lenguaje máquina SISA (en hexadecimal)	**Lenguaje ensamblador SISA**
0x637F	STB -1(R1), R5
0x7007	JALR R0, R0
0x0897	`SHL R2, R4, R2`
0xA7F0	`OUT 0xF0, R3`

Ejercicio 3 (2 puntos) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). (0,5 por cada estado correcto)

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
D	`JALR R3, R5`	0x0020B0
Addr	`ST 0x10(R5), R3`	0x000030
In	`IN R3, 0`	0x140802
Bnz	`BNZ R7, 4`	0x800220

Ejercicio 4 (1 punto) Indicad qué cambios se producen en el estado del computador **SISC Von Neumann** después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale 0x1008, el contenido de todos los registros pares es 0x2468 y el de los registros impares es 0xF000, el byte contenido en todas las direcciones pares de la memoria es 0x22 y el de todas las impares es 0x11 y el contenido de los ports KEY_DATA y KEY_STATUS es 1. Utilizad la notación MEMb[0x…]=0x... y/o MEMw[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida. (0,25 por cada instrucción. El PC=PC+2 sólo se cuenta mal una vez si es el mismo error)

Instrucción a ejecutar	**Cambios en el estado del computador**
`JALR R3, R5`	PC = 0xF000, R3 = 0x100A
`ST 0x10(R5), R3`	PC = 0x100A, Mem_w[0xF010] = 0xF000
`IN R3, KEY_DATA`	PC = 0x100A, R3 = 1, PORTIN[KEY_STATUS] = 0
`BNZ R7, 4`	PC = 0x1012

Apellidos: ……Nombre:……..Grupo:……DNI:……

Ejercicio 5 (1 punto) Completad las filas y columnas de la siguiente tabla que representa un subconjunto de la palabra de control del SISC Von Neumann. Indicad solo los valores necesarios para realizar las acciones correspondientes a cada estado. Para el resto de valores, ya sean desconocidos o no sean necesarios para ninguna acción del estado, poned x. (-0,25 por cada fila/columna con error minimizando el número de fials/columnas que cubren todos los errores. No puede ser menor a 0)

Nodo / Estado	**Instrucción en IR** (en ensamblador)	@A	@B	@D	F	N (en hexadecimal)	ADDR-IO (en hexadecimal)
D	`MOVHI R3, 0xFF`	011	111	xxx	100	0xFFFE	XX
Addr	`ST 0x10(R5), R3`	xxx	011	xxx	100	0x0010	XX
In	`IN R3, 0`	xxx	xxx	011	xxx	XXXX	00
Bnz	`BNZ R7, 4`	xxx	xxx	xxx	000	XXXX	XX

Ejercicio 6 (1 punto) Especificad el camino crítico (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el tiempo de ciclo mínimo para que el computador SISC Von Newmann pueda ejecutar correctamente los siguientes estados. No debéis añadir ningún porcentaje de seguridad al cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

Tp**(ROM_Q+)** = 50 u.t.
Tp**(ROM_OUT)** = 70 u.t.
Tp**(MUX-2-1)** = 50 u.t.
Tp**(MUX-4-1)** = 100 u.t.
Tp**(REG)** = 100 u.t. // Tp de un registro.
Tp**(REGFILE)** = 250 u.t. // Tiempo de lectura/escritura de un registro.
Tp**(ALU-slow)** = 700 u.t. // Tp de ALU para operaciones lentas: ADD, SUB, CMP*.
Tp**(ALU-quick)** = 350 u.t. // Tp de ALU para operaciones rápidas: todas menos las anteriores.
Tacc**(64Kb MEMORY)** = 900 u.t. // Tiempo de acceso (lectura o escritura) a la memoria. Fijaros que este tiempo incluye toda la lógica empleada para construir-la a partir de dos bloques de 32Kb.
Tp**(AND-2)** = Tp**(OR-2)** = 20 u.t.
Tp**(NOT)** = 10 u.t.

Desconocemos como se han implementado los bloques internamente (y podría ser diferente a como los hemos visto en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os facilitamos su esquema porque lo deberíais saber).

a. F (Fetch) = (0,25 el camino, 0,25 el tiempo, sólo si el camino es correcto)

100+70+50+900+50 (REG->ROM_OUT->MUX2-1 ->MEMORY->MUX2-1->IR) = 1170 ut

b. Addr = (0,25 el camino, 0,25 el tiempo, sólo si el camino es correcto)

100+70+100+50+700 (REG->ROM_OUT->MUX4-1->MUX2-1->ALU-slow->R@) = 1020 ut

Ejercicio 7 (2 puntos)

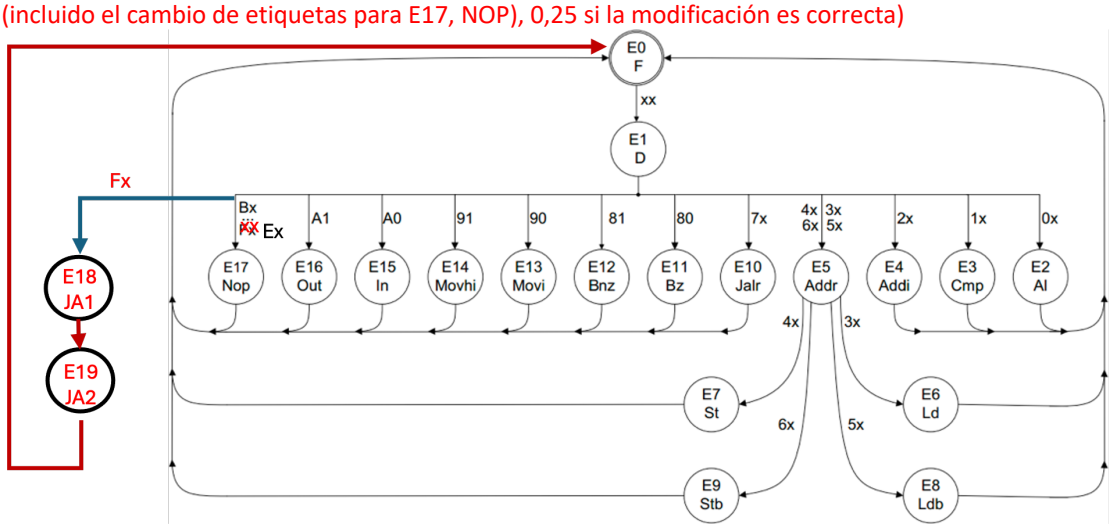
El diseñador del lenguaje máquina SISA considera que es preciso añadir una nueva instrucción al repertorio de instrucciones. Se trata de una versión del JALR condicional que únicamente saltará y guardará el valor del PC si Rb es 0 (de forma similar a como lo hace el BZ).

Sintaxis: JALRZ Rd, Ra, Rb

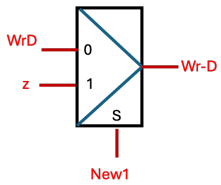
Codificación: 1111 aaa bbb ddd 111

Semántica: PC = PC + 2; if (Rb == 0) {PC = Ra; Rd = PC}

a. ¿Cómo modificaríais el hardware de la UPG para poder implementar esta nueva instrucción? Podéis modificar buses y añadir multiplexor(es), buses y señales de control, pero no podéis modificar ni los bloques ni el uso de las señales de control existentes. Describe las modificaciones con un dibujo de la parte relevante o mediante una explicación. Indicad también como cambiaria el grafo de estados. (0,25 si el grafo es correcto (incluido el cambio de etiquetas para E17, NOP), 0,25 si la modificación es correcta)



Añadimos una señal (New1) a la ROM_OUT que valdrá 1 en el estado JA2 y 0 en todos los otros estados. Conectamos esta nueva señal como selector del siguiente multiplexor (WrD es la señal que sale de la ROM_OUT que teníamos hasta ahora y Wr-D es la que se conectará al banco de registros):



b. Indicad las acciones que se deberán ejecutar en cada uno de los estados de la nueva instrucción (**Sin incluir Fetch y decode**). La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean todas necesarias. (0,25 por cada estado correcto, binario)

Estado	Acciones
JA1	R@ = Rx// Ry = Rb
JA2	si (Ry == 0) {PC=R@, Rd=PC}

c. Indicad el contenido de las filas de la ROM_OUT que sea preciso modificar (**sin incluir Fetch y decode**). La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean todas necesarias. También hemos añadido un par de salidas nuevas por si las necesitas. De nuevo, puede que no las necesites todas. (por cada fila: 0,5 si es correcta, 0,25 si sólo hay un error, 0 si hay más de un error)

@ROM	New1	New2	Bnz	Bz	WrMem	RdIn	WrOut	WrD	Ldlr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A1	P/I/L/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0
JA1	0		0	0	0	0	0	0	0	x	x	x	0	x	x	x	1	0	x	x	1	0	0	0	x	x
JA2	1		0	1	0	0	0	x	x	x	x	0	x	1	1	1	1	0	x	x	1	0	0	1	0	0